

2. Requirement Analysis Phase

Project: PocketSmart AI - Smart Budget & Recommendation Assistant

2.1 Introduction

The Requirement Analysis Phase identifies what the PocketSmart AI system must provide in order to solve the problem defined during the brainstorming stage. The project is intended to generate personalized, budget-aware recommendations for home interiors, party planning, and jewelry selection. The requirements therefore cover user inputs, AI processing, recommendation generation, authentication, frontend interaction, and backend services.

2.2 Purpose of the System

The main requirement is to create an intelligent platform that can understand a user's budget, preferences, and contextual needs and then produce suitable recommendations. The project documentation describes the system as a cross-platform recommendation solution that can connect different product and service ecosystems.

2.3 Target User Needs

Users need a simple interface for entering their requirements. They should be able to select an appropriate planner and provide only the information needed for that planning task. The system should then process the information and present structured recommendations that are easy to understand.

2.4 Major Functional Areas

The required functional areas are Home Interior Planning, Party Planning, Jewelry Planning, User Registration and Login, Session Handling, Recommendation History, and AI-based recommendation generation. These areas form the core scope of the application.

2.5 Requirement Analysis Outcome

At the end of this phase, the project has a clearer definition of the features, inputs, technologies, and processing flow required for implementation. The requirements also provide a foundation for designing the FastAPI backend, planner forms, Gemini integration, and recommendation display pages.

Page 2 - Functional Requirements

2.6 Home Interior Planner Requirements

The Home Interior Planner must accept a budget and home-related details such as room type and item quantities. The project documentation gives examples including Living Room, Kitchen, and Bedroom, along with items such as lights, ceiling fans, and dining tables.

After receiving the input, the system must process the requirements and generate cost-effective recommendations. The recommendation should consider factors such as functionality, style, and price while remaining within the user's defined financial plan.

2.7 Party Planner Requirements

The Party Planner must collect the total budget, guest count, event type, and venue-related information. The system should use these inputs to organize recommendations for catering, decoration, entertainment, and, where applicable, accommodation.

The planner should support different event types such as birthday, corporate, and wedding events. The requirement is not simply to provide generic suggestions, but to tailor them to the user's event context and budget.

2.8 Jewelry Planner Requirements

The Jewelry Planner must accept a budget, occasion, and style preferences. It should also support an optional outfit image upload. The image can be used by the AI to consider color coordination and aesthetics.

The system should return jewelry options that are matched to the occasion and style and are aligned with the user's financial plan. This requirement creates a multimodal input scenario involving both text and image data.

2.9 Recommendation Requirements

Recommendations must be structured and readable. The project UI specification calls for card-like layouts that can display product suggestions, item details, prices, and other useful recommendation information.

2.10 Authentication Requirements

The system must support user registration, login, logout, token-based authorization, and protected access to applicable routes. The documented routes include `/register`, `/login`, `/logout`, and `/token`.

Page 3 - Input, AI, and Backend Requirements

2.11 User Input Requirements

User inputs must be collected according to the selected planner. Budget is a central input for all three planning domains. Additional inputs vary by domain: home planning requires room and item information; party planning requires event and guest information; jewelry planning requires occasion, style, and optional image input.

2.12 AI Integration Requirements

The project requires an AI layer capable of understanding budget context, generating personalized suggestions, and processing multimodal input where required. The project documentation identifies Gemini 1.5 Flash Pro as the planned AI model for these functions.

The AI integration should use planner-specific prompts and templates. The workflow also identifies prompt fine-tuning for budget interpretation and recommendation generation, as well as image analysis for the Jewelry Planner.

2.13 Backend Requirements

The backend must provide modular request handling and communication between the frontend and Gemini AI layer. The documented implementation uses FastAPI and defines routes for the major planning and user-management functions.

2.14 Required API Routes

The planned routes include /generate-home for home recommendations, /generate-party for party recommendations, and /generate-jewelry for jewelry recommendations. Authentication routes include /register, /login, /logout, and /token. Additional routes such as /session-info, /session-data, /recommendations-details, /history, and /startup support session information, detailed results, history, and application initialization.

2.15 Data and Session Requirements

Session-related functionality is required so that the application can identify login status and retain session-specific data for personalization and recommendation tracking. The history feature should allow users to review previous recommendation queries and results.

2.16 Configuration Requirements

The project requires secure configuration of the AI API key and environment settings. The workflow specifies setting up API access, configuring permissions and usage limits, and loading environment variables before the backend starts.

Page 4 - Frontend and Non-Functional Requirements

2.17 Frontend Requirements

The frontend must provide a lightweight, responsive, and intuitive web interface. The project documentation specifies HTML, CSS, JavaScript, and Jinja2 templates for collecting inputs and displaying AI-generated results.

2.18 Planner Form Requirements

Separate forms are required for Home Interior, Party, and Jewelry planning. Each form must contain fields appropriate to the selected planning task. The Jewelry form must additionally support an optional image upload.

2.19 Result Display Requirements

AI responses should be presented in a clear structured format rather than as an unformatted block of text. The documented interface calls for card-like displays containing recommendation information such as product suggestions, item details, and prices.

2.20 Responsiveness and Usability

The interface should work smoothly across devices and should provide a clean user experience. The project documentation specifically identifies responsive UI, dynamic result display, and easy interaction with the different planner modules as development requirements.

2.21 Security and Access Requirements

User authentication and protected routes are required for personalized features. The system should maintain session control and use token-based authentication for authorized access to protected API routes.

2.22 Reliability Requirements

The application should validate user inputs and handle edge cases so that invalid or incomplete data does not cause failures. The testing and optimization workflow also identifies fallback or default recommendations when the AI returns insufficient results.

2.23 Performance and Maintainability

The backend should use a modular architecture with separate areas for routes, services, and models. The AI recommendation logic is intended to be organized so that prompts, budget formatting, domain segmentation, and image analysis can be maintained without tightly coupling all features together.

Page 5 - External Integration, Testing, and Final Requirements

2.24 External Platform Requirements

The project concept requires access to product and service information from multiple platforms. The documentation mentions Amazon, Flipkart, IKEA, Swiggy, Zomato, and OYO as example sources. During development, the workflow allows mock API calls or simulated sourcing where direct integrations are not available.

2.25 Testing Requirements

The system must be tested with real-world budget inputs across all three planning domains. Testing should cover home decor recommendations, party packages involving food, venues, and decoration, and jewelry suggestions based on outfit styles, occasions, and optional images.

2.26 AI Quality Requirements

The quality of Gemini-generated responses must be evaluated for relevance, budget adherence, contextual awareness, and platform accuracy. Prompt structures may need to be refined when results do not sufficiently match user requirements.

2.27 Validation Requirements

Input validation is required to reduce failures caused by edge cases. The system should also provide fallback or default recommendations when the AI cannot produce enough suitable results.

2.28 Expected Pages and User Flow

The requirement analysis also covers the main user-facing pages: Home Page, Register Page, Login Page, User Dashboard, Home Interior Planner and Recommendations pages, Party Planner and Recommendations pages, Jewelry Planner and Recommendations pages, Testimonials Page, and Recommendation History Page.

2.29 Final Requirement Summary

The Requirement Analysis Phase establishes the functional and technical needs of PocketSmart AI. The system must collect budget and contextual information, support three specialized planners, communicate with a Gemini-based AI recommendation layer, provide authentication and session management, display structured results, maintain history, and support testing and fallback behavior.

2.30 Conclusion

These requirements provide the foundation for the next development stages. They define what the application must do, what information it must process, which major routes and interfaces are needed, and how the system should be tested for reliable budget-aware recommendations.